



**TRIBHUVAN UNIVERSITY  
INSTITUTE OF ENGINEERING  
PURWANCHAL CAMPUS**

**A MINOR PROJECT REPORT  
ON  
“ DESIGN AND IMPLEMENTATION OF A TWO-WHEELED  
SELF-BALANCING ROBOT WITH OPTIMIZED PID CONTROL”**

**BY:**

**AADESH DAHAL (PUR080BEI002)  
NISHCHAL POKHREL (PUR080BEI027)  
PRASHAN CHENTA RAI (PUR080BEI030)  
YOGESH KHADKA (PUR080BEI048)**

**DEPARTMENT OF ELECTRONICS AND COMPUTER  
ENGINEERING  
PURWANCHAL CAMPUS  
DHARAN, NEPAL**

**AUGUST,2026**

**DESIGN AND IMPLEMENTATION OF A TWO-WHEELED  
SELF-BALANCING ROBOT WITH OPTIMIZED PID CONTROL**

By:

AADESH DAHAL (PUR080BEI002)

NISHCHAL POKHREL (PUR080BEI027)

PRASHAN CHENTA RAI (PUR080BEI030)

YOGESH KHADKA (PUR080BEI048)

A project submitted to the Department of Electronics and Computer Engineering in  
partial fulfillment of the requirements for the Bachelor's Degree in Computer  
Engineering

Department of Electronics and Computer Engineering  
Purwanchal Campus, Institute of Engineering  
Tribhuvan University  
Dharan, Nepal

August, 2026

## **COPYRIGHT©**

This project report has been prepared as part of the academic requirements of the Department of Electronics and Computer Engineering, Institute of Engineering, Purwanchal Campus. The report may be consulted and used for educational, research, and scholarly purposes with proper acknowledgement of the authors and the institution.

Permission to reproduce a substantial portion of this report, in print or electronic form, must be obtained from the authors or the Department of Electronics and Computer Engineering. Any reproduction, distribution, publication, or commercial use of the contents without prior written authorization is prohibited.

Request for permission to copy or make any other use of the material in this report in whole or in part should be addressed to:

Head

Department of Electronics and Computer Engineering

Purwanchal Campus, Institute of Engineering,

Dharan, Sunsari

Nepal

## DECLARATION

We declare that the minor project report entitled “**DESIGN AND IMPLEMENTATION OF A TWO-WHEELED SELF-BALANCING ROBOT WITH OPTIMIZED PID CONTROL**”, submitted to the Department of Electronics and Computer Engineering, Institute of Engineering, Purwanchal Campus, in partial fulfillment of the requirements for the Bachelor’s Degree in Electronics, Communication and Information Engineering, is an original record of the project work carried out by our group.

We further declare that this report has not previously been submitted, either in whole or in part, to any university or academic institution for the award of a degree or other qualification. All information, ideas, and materials obtained from external sources have been properly acknowledged and cited in the report. We accept full responsibility for the originality and authenticity of the work presented herein.

AADESH DAHAL (PUR080BEI002)

NISHCHAL POKHREL (PUR080BEI027)

PRASHAN CHENTA RAI (PUR080BEI030)

YOGESH KHADKA (PUR080BEI048)

August, 2026

## ACKNOWLEDGEMENT

The authors express their sincere gratitude to the Department of Electronics and Computer Engineering, Purwanchal Campus, Institute of Engineering, Tribhuvan University, for providing them with the opportunity and necessary resources to undertake the minor project entitled “**DESIGN AND IMPLEMENTATION OF A TWO-WHEELED SELF-BALANCING ROBOT WITH OPTIMIZED PID CONTROL**”.

The authors would also like to extend our sincere appreciation to **Asst. Prof. Manoj Kumar Guragai**, Head of the Department of Electronics and Computer Engineering, as well as all the faculty members and laboratory staff who directly or indirectly supported us during the project.

The authors also acknowledge the respected faculty and teachers for their insightful feedback and academic support, which contributed significantly to the improvement of this project.

Finally, the authors would like to thank their friends and peers for their encouragement and assistance during the course of this project.

AADESH DAHAL (PUR080BEI002)

NISHCHAL POKHREL (PUR080BEI027)

PRASHAN CHENTA RAI (PUR080BEI030)

YOGESH KHADKA (PUR080BEI048)

## ABSTRACT

A self-balancing robot is an inherently unstable system that requires continuous sensing and real-time feedback control to maintain its upright position. This project presents the design and development of a two-wheeled Self-Balance Robot using an ESP32 DevKit V1 as the main processing and control unit. The system uses an MPU6050 inertial measurement unit to measure the robot's tilt angle and angular velocity. The accelerometer and gyroscope readings are processed to obtain a stable estimate of the robot's orientation.

A Proportional-Integral-Derivative (PID) controller continuously calculates the difference between the measured tilt angle and the desired balance angle. Based on this error, the ESP32 generates appropriate control signals for two L298N motor drivers. Each motor driver independently controls a JGB37-520 12 V, 150 RPM DC geared motor equipped with a quadrature encoder. The motors provide the corrective movement required to maintain balance, while the encoders provide rotational feedback that can be used to monitor motor speed, direction, and position.

The robot is powered by a LiPo battery, and a buck converter is used to provide suitable regulated voltage to the electronic components. The mechanical chassis is constructed using custom 3D-printed parts, allowing the components to be securely mounted with appropriate alignment and weight distribution. This project demonstrates the practical application of PID control, sensor interfacing, encoder feedback, embedded systems, motor control, and mechanical design. It also provides a foundation for future improvements such as wireless monitoring, remote control, position control, obstacle detection, and autonomous navigation.

**Keywords:** Self-balance robot, ESP32 DevKit V1, PID controller, MPU6050, L298N motor driver, JGB37-520 geared motor, quadrature encoder, LiPo battery, buck converter, 3D-printed chassis.

# TABLE OF CONTENTS

|                                           |            |
|-------------------------------------------|------------|
| <b>COPYRIGHT</b>                          | <b>i</b>   |
| <b>DECLARATION</b>                        | <b>ii</b>  |
| <b>ACKNOWLEDGEMENT</b>                    | <b>iii</b> |
| <b>ABSTRACT</b>                           | <b>iv</b>  |
| <b>LIST OF FIGURES</b>                    | <b>ix</b>  |
| <b>LIST OF TABLES</b>                     | <b>x</b>   |
| <b>1 INTRODUCTION</b>                     | <b>1</b>   |
| 1.1 Background . . . . .                  | 1          |
| 1.2 Self-Balancing Robot . . . . .        | 2          |
| 1.3 Problem Statement . . . . .           | 2          |
| 1.4 Objectives . . . . .                  | 3          |
| 1.5 Application . . . . .                 | 3          |
| <b>2 LITEERATURE REVIEW</b>               | <b>4</b>   |
| <b>3 SYSTEM DESIGN</b>                    | <b>5</b>   |
| 3.1 Overall System Architecture . . . . . | 5          |
| 3.2 Working Principle . . . . .           | 5          |
| 3.3 Hardware Design . . . . .             | 6          |
| 3.4 Software Design . . . . .             | 7          |
| 3.4.1 Software Implementation . . . . .   | 7          |
| 3.5 PID Control Algorithm . . . . .       | 9          |
| 3.6 Angle Estimation Method . . . . .     | 10         |

|          |                                                |           |
|----------|------------------------------------------------|-----------|
| 3.7      | Circuit Diagram . . . . .                      | 11        |
| 3.8      | Flowchart . . . . .                            | 12        |
| <b>4</b> | <b>SYSTEM IMPLEMENTATION</b>                   | <b>13</b> |
| 4.1      | Hardware Implementation . . . . .              | 13        |
| 4.2      | Sensor Interfacing . . . . .                   | 13        |
| 4.3      | Motor Driver Interfacing . . . . .             | 14        |
| 4.4      | Software Implementation . . . . .              | 14        |
| 4.4.1    | Balance PID Controller . . . . .               | 15        |
| 4.4.2    | Balance and Safety Conditions . . . . .        | 16        |
| 4.4.3    | Motor Direction Control . . . . .              | 17        |
| 4.4.4    | Left and Right Motor Synchronization . . . . . | 18        |
| 4.4.5    | Automatic Motor Trim . . . . .                 | 18        |
| 4.4.6    | Startup Ramp . . . . .                         | 19        |
| 4.4.7    | PWM Slew Limiting . . . . .                    | 19        |
| 4.4.8    | MPU6050 Angle Estimation . . . . .             | 20        |
| 4.4.9    | Overall Software Control Flow . . . . .        | 21        |
| 4.5      | System Integration . . . . .                   | 21        |
| 4.6      | Testing and Validation . . . . .               | 21        |
| 4.6.1    | MPU6050 Calibration and Angle Test . . . . .   | 22        |
| 4.6.2    | Complementary Filter Test . . . . .            | 22        |
| 4.6.3    | PID Balance Test . . . . .                     | 23        |
| 4.6.4    | Encoder and RPM Test . . . . .                 | 23        |
| 4.6.5    | Motor Synchronization Test . . . . .           | 24        |



|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| 4.6.6    | Safety Test . . . . .                                            | 24        |
| 4.6.7    | Final Balancing Test . . . . .                                   | 25        |
| <b>5</b> | <b>Results and Discussion</b>                                    | <b>26</b> |
| 5.1      | System Architecture: RTOS and WebSocket Integration . . . . .    | 26        |
| 5.2      | Balancing Stability and Steady-State Performance . . . . .       | 26        |
| 5.3      | Real-Time Data Visualization and Disturbance Rejection . . . . . | 28        |
| 5.4      | Motor Synchronization and PWM Analysis . . . . .                 | 29        |
| 5.5      | Summary and Recommendations . . . . .                            | 29        |
| <b>6</b> | <b>Conclusion</b>                                                | <b>31</b> |
| 6.1      | Conclusion . . . . .                                             | 31        |
| 6.2      | Major Achievements . . . . .                                     | 31        |
| 6.3      | Control System Performance and Critical Analysis . . . . .       | 32        |
| 6.4      | System Limitations . . . . .                                     | 32        |
| 6.5      | Future Scope . . . . .                                           | 33        |
| 6.5.1    | PID Retuning and Filtering . . . . .                             | 33        |
| 6.5.2    | Adaptive PID Control . . . . .                                   | 33        |
| 6.5.3    | Advanced State Estimation . . . . .                              | 33        |
| 6.5.4    | Position and Velocity Control . . . . .                          | 33        |
| 6.5.5    | Autonomous Navigation . . . . .                                  | 33        |
| 6.5.6    | Data Logging . . . . .                                           | 34        |
| 6.5.7    | Mechanical Improvements . . . . .                                | 34        |
| 6.6      | Final Remarks . . . . .                                          | 34        |
|          | <b>REFERENCES</b>                                                | <b>35</b> |

|          |                                       |           |
|----------|---------------------------------------|-----------|
| <b>A</b> | <b>Project Overview</b>               | <b>36</b> |
| <b>B</b> | <b>Pin Mapping and Wiring Diagram</b> | <b>38</b> |
| <b>C</b> | <b>Main Code</b>                      | <b>40</b> |

## LIST OF FIGURES

|                                                                                                                                                                        |    |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Figure 1.1: Inverted pendulum model representing the self-balancing robot. . . . .                                                                                     | 1  |
| Figure 1.2: Block diagram of the self-balancing robot system. . . . .                                                                                                  | 2  |
| Figure 3.1: Block diagram of self-balanancing robot architecture . . . . .                                                                                             | 5  |
| Figure 3.2: Complete system circuit schematic featuring dual independent<br>motor drivers, logic level power distribution, and microcontroller pin<br>mapping. . . . . | 11 |
| Figure 3.3: Flowchart of the main control loop for the self-balancing robot . . . . .                                                                                  | 12 |
| Figure 4.1: Block diagram of control sequence . . . . .                                                                                                                | 15 |
| Figure 4.2: software control flowchart . . . . .                                                                                                                       | 21 |
| Figure 4.3: Complete feedback sequence of final validation . . . . .                                                                                                   | 25 |
| Figure 5.1: Pitch Response . . . . .                                                                                                                                   | 27 |
| Figure 5.2: PID term contributions streamed via WebSocket, showing de-<br>structive D-term noise amplification. . . . .                                                | 28 |
| Figure 5.3: Filtered pitch angle detail, highlighting the high-frequency jitter<br>perfectly captured by the RTOS loop. . . . .                                        | 29 |
| Figure 6.1: Block Diagram of Control Loop . . . . .                                                                                                                    | 31 |
| Figure 6.2: Block Diagram of Position and Velocity Control . . . . .                                                                                                   | 33 |

## LIST OF TABLES

|                                                                          |    |
|--------------------------------------------------------------------------|----|
| Table 3.1: Hardware Components and Their Functions . . . . .             | 6  |
| Table 4.1: MPU6050 to ESP32 connection . . . . .                         | 13 |
| Table 4.2: L298N to ESP32 connection . . . . .                           | 14 |
| Table 5.1: Current PID gains and system communication rates. . . . .     | 26 |
| Table A.1: Project Cost Breakdown . . . . .                              | 36 |
| Table A.2: MPU6050 Specifications . . . . .                              | 36 |
| Table A.3: L298N Specifications . . . . .                                | 37 |
| Table A.4: DC Gear Motor Specifications . . . . .                        | 37 |
| Table B.1: Complete GPIO pin mapping for the ESP32 self-balancing robot. | 38 |

## **LIST OF ABBREVIATIONS**

|        |                                            |
|--------|--------------------------------------------|
| 2S     | : Two Cells (in Series)                    |
| ABS    | : Acrylonitrile Butadiene Styrene          |
| AIN    | : A-channel Input                          |
| BIN    | : B-channel Input                          |
| Config | : Configuration                            |
| DC     | : Direct Current                           |
| ESP32  | : Espressif Systems 32-bit Microcontroller |
| F-F    | : Female-to-Female                         |
| Fig.   | : Figure                                   |
| GND    | : Ground                                   |
| GPIO   | : General Purpose Input/Output             |
| Gyro   | : Gyroscope                                |
| HTTP   | : Hypertext Transfer Protocol              |
| Hz     | : Hertz                                    |
| I2C    | : Inter-Integrated Circuit                 |
| IDE    | : Integrated Development Environment       |
| IMU    | : Inertial Measurement Unit                |
| Init   | : Initialize                               |
| IP     | : Internet Protocol                        |
| ISR    | : Interrupt Service Routine                |
| Kc     | : Critical Gain                            |
| Kd     | : Derivative Gain                          |
| Ki     | : Integral Gain                            |
| Kp     | : Proportional Gain                        |
| kHz    | : kilohertz                                |
| L298N  | : Dual H-Bridge Motor Driver               |
| LEDC   | : LED Control (ESP32 peripheral)           |
| Li-ion | : Lithium-ion                              |

|                |                                                             |
|----------------|-------------------------------------------------------------|
| LiPo           | : Lithium Polymer                                           |
| LQR            | : Linear Quadratic Regulator                                |
| M-F            | : Male-to-Female                                            |
| MEMS           | : Micro-Electro-Mechanical Systems                          |
| MPU            | : Motion Processing Unit                                    |
| MPU6050        | : 6-axis Motion Tracking Sensor (Inertial Measurement Unit) |
| ms             | : milliseconds                                              |
| PCB            | : Printed Circuit Board                                     |
| PID            | : Proportional-Integral-Derivative                          |
| PWM            | : Pulse Width Modulation                                    |
| PWMA           | : Pulse Width Modulation Channel A                          |
| PWMB           | : Pulse Width Modulation Channel B                          |
| RPM            | : Revolutions Per Minute                                    |
| SCL            | : Serial Clock                                              |
| SDA            | : Serial Data                                               |
| SPST           | : Single-Pole Single-Throw                                  |
| Sync           | : Synchronization                                           |
| Tab.           | : Table                                                     |
| T <sub>c</sub> | : Oscillation Period                                        |
| Trim           | : Trimming                                                  |
| TWSBR          | : Two-Wheeled Self-Balancing Robot                          |
| VCC            | : Voltage Common Collector                                  |
| VM             | : Motor Voltage                                             |
| Wi-Fi          | : Wireless Fidelity                                         |

# CHAPTER 1

## INTRODUCTION

### 1.1 Background

Self-balancing robots are a class of mobile robots that maintain an upright posture on two wheels by continuously correcting their tilt angle. They are based on the inverted pendulum principle, where the centre of mass is above the pivot point, making the system inherently unstable without active control. The development of such robots has gained significant interest due to their applications in personal transportation (e.g., Segway), humanoid robotics, and as educational platforms for learning control system design.

With the advancement of low-cost microcontrollers and micro-electromechanical systems (MEMS) sensors, building a self-balancing robot has become feasible for academic and hobbyist projects. The ESP32 microcontroller, combined with the MPU6050 inertial measurement unit (IMU), provides a powerful yet affordable platform for implementing real-time balancing algorithms. The main challenge lies in accurately estimating the tilt angle and designing a robust control law that can maintain balance despite sensor noise and external disturbances.

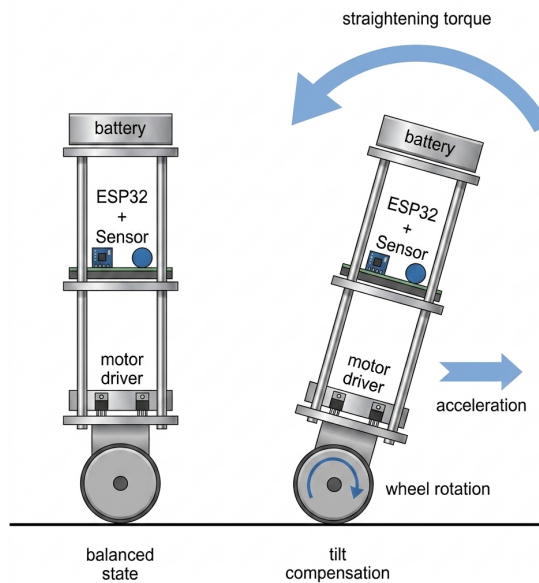


Figure 1.1: Inverted pendulum model representing the self-balancing robot.

## 1.2 Self-Balancing Robot

A typical self-balancing robot consists of a microcontroller, an inertial measurement unit (IMU) such as the MPU6050, a motor driver, DC motors with encoders, and a battery system. The IMU measures the tilt angle and angular velocity of the robot body. The microcontroller processes these sensor readings and implements a control algorithm, usually a Proportional-Integral-Derivative (PID) controller, to compute the required motor command. The motor driver then delivers the necessary voltage to the DC motors to keep the robot upright.

The robot can be modelled as an inverted pendulum mounted on a moving cart. When the robot tilts forward, the controller drives the wheels forward to move the base under the centre of mass, thereby restoring the upright position. This continuous correction loop is what enables the robot to balance on two wheels.

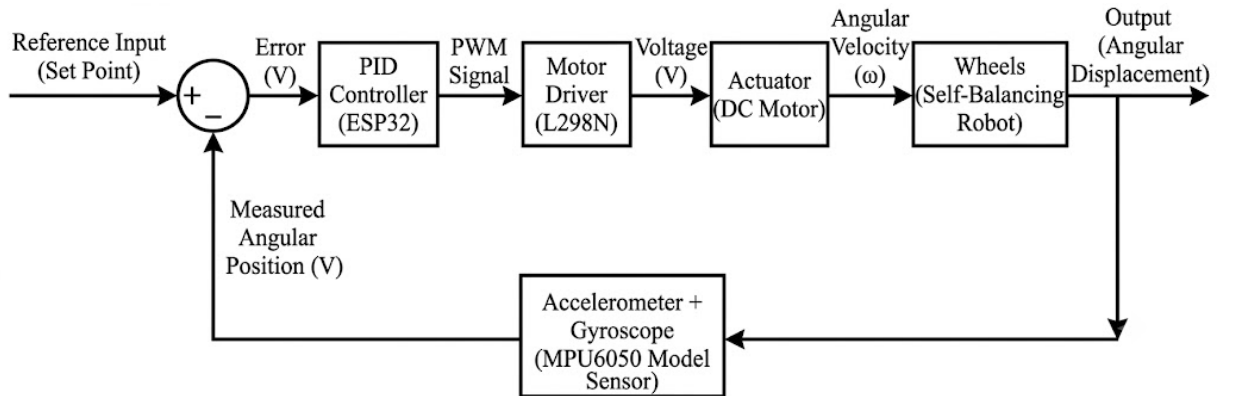


Figure 1.2: Block diagram of the self-balancing robot system.

## 1.3 Problem Statement

Despite the commercial availability of self-balancing vehicles, designing and implementing a low-cost, open-source self-balancing robot using ESP32 and MPU6050 remains challenging. The main difficulties include:

- Obtaining a reliable tilt angle from noisy IMU data.
- Designing an appropriate PID controller that can handle motor nonlinearities and external disturbances.



- Tuning the PID gains to achieve stable and responsive balancing.
- Integrating the hardware and software components into a compact, functional system.

Therefore, the problem addressed in this project is to design, build, and validate a self-balancing robot that can maintain its upright position and recover from disturbances using a PID control strategy implemented on an ESP32 microcontroller.

## 1.4 Objectives

The main objectives of this project are:

- To study the dynamics of an inverted pendulum and the working principle of a self-balancing robot.
- To design and assemble the hardware system using ESP32, MPU6050, motor driver, DC motors, and a suitable battery system.
- To implement an angle estimation algorithm (complementary filter) for accurate tilt measurement.
- To implement and tune a PID controller for real-time balance control.
- To test and evaluate the performance of the robot in terms of balancing, disturbance rejection, and system stability.

## 1.5 Application

The application of self balancing robot are given below:

- Personal transportation devices such as self-balancing scooters and Segways.
- Autonomous delivery robots for transporting parcels, food, and medical supplies.
- Mobile robotic platforms used in warehouses and industrial automation systems.
- Healthcare and rehabilitation assistance for elderly and physically challenged individuals.
- Educational and research platforms for studying control systems, robotics, and embedded systems.

## CHAPTER 2

### LITEERATURE REVIEW

Two-wheeled self-balancing robots (TWSBRs) are underactuated, nonlinear systems that require continuous feedback control. This review summarises the most relevant works for a low-cost, ESP32-based self-balancing robot using PID control.

A comprehensive review of control strategies for TWSBRs is presented in [1], covering classical PID, LQR, nonlinear methods, and intelligent techniques. It provides practical guidelines for selecting controllers based on computational resources and robustness, confirming that a well-tuned PID controller is an excellent choice for embedded real-time applications.

A rigorous evaluation of PID control for a TWSBR is given in [2], where the controller reaches dynamic balance in under 1.2 seconds. The maximum tolerable disturbance angle is quantified as  $\theta = 0.0524$  rad, and stability is maintained under sudden payload changes, demonstrating robustness.

A complete ESP32-based implementation is documented in [3], using an MPU6050 IMU and PID gains  $K_P = 6.0$ ,  $K_I = 0.1$ ,  $K_D = 1.5$ . It achieves tilt variations of  $\pm 2.5^\circ$  with a hardware cost of USD 127, including gyro drift compensation and dual-mode control.

Practical sensor fusion using a Kalman filter on an ESP32 is demonstrated in [4] by reverse-engineering a hoverboard. The platform autonomously balances, carries payloads, and operates with high energy efficiency, offering a sustainable low-cost solution.

An educational PID-based self-balancing robot using an ESP32 is presented in [5], detailing sensor calibration and PID tuning methods. The robot processes MPU6050 angle signals to drive DC motors for smooth, balanced motion, serving as a practical guide for implementation.

In summary, these five works provide a complete foundation: theoretical overview, validated PID design, ESP32 implementation metrics, Kalman-filter sensor fusion, and practical tuning procedures. An optimised PID controller with robust sensor fusion remains the most practical choice for this project.

## CHAPTER 3

### SYSTEM DESIGN

#### 3.1 Overall System Architecture

The self-balancing robot is designed as a compact two-wheeled platform that maintains its upright posture through real-time feedback control. The system architecture comprises four main functional blocks: sensing, processing, actuation, and power management. The MPU6050 inertial measurement unit (IMU) serves as the primary sensor for measuring the robot's tilt angle and angular velocity. The ESP32 microcontroller acts as the central processing unit, executing the control algorithm and generating pulse-width modulation (PWM) signals. A motor driver (L298N) interfaces the microcontroller with two DC motors, providing the necessary current and voltage to the wheels. The power system includes a rechargeable battery pack and voltage regulators to supply stable voltage to all components.

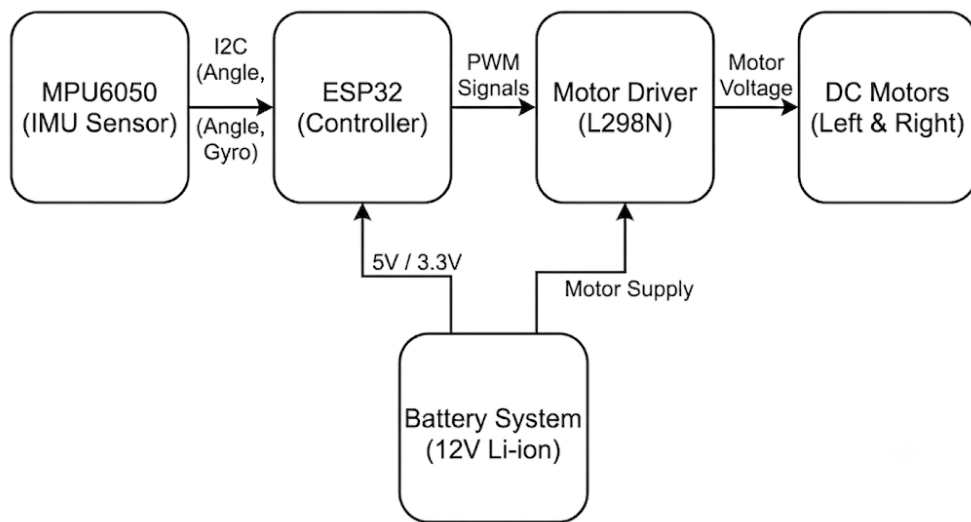


Figure 3.1: Block diagram of self-balancing robot architecture

#### 3.2 Working Principle

The self-balancing robot operates on the principle of an inverted pendulum. When the robot tilts forward, the centre of mass moves ahead of the wheel axle, creating a torque that would cause the robot to fall. To counteract this, the controller drives the wheels

forward, moving the base underneath the centre of mass and restoring the vertical posture. This process is continuous: the IMU measures the tilt angle and angular velocity at a high rate (e.g., 100 Hz), the ESP32 processes these measurements to compute the control output, and the motors are commanded to move in the direction of the tilt. A well-tuned control loop ensures the robot remains balanced even under small external disturbances.

### 3.3 Hardware Design

The hardware design focuses on selecting cost-effective, easily available components that meet the performance requirements of real-time balancing. The following subsections describe each major component.

Table 3.1: Hardware Components and Their Functions

| Component                                   | Function                                                                                                                                                                                                                   |
|---------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ESP32 DevKitC V4                            | Acts as the main controller of the robot. It executes the balancing algorithm, processes sensor data from the MPU6050 via I <sup>2</sup> C communication, and generates PWM signals to control the motor driver.           |
| MPU6050 IMU                                 | Measures the robot's motion using a 3-axis accelerometer and 3-axis gyroscope. Provides acceleration and angular velocity data that are fused using a complementary filter to estimate the tilt angle for balance control. |
| L298N Motor Driver                          | Interfaces the ESP32 with the DC motors. Receives PWM and direction signals from the microcontroller to provide efficient bidirectional speed and direction control of the motors.                                         |
| JGB37-520 DC Geared Motor                   | Generate the mechanical motion required for balancing and movement. The geared motors provide sufficient torque, while the integrated magnetic encoders can supply rotational feedback for future control enhancements.    |
| 3-Cell Li-ion Battery Pack (12 V)           | Serves as the primary power source for the robot. Supplies energy to the motors and electronics through voltage regulation circuitry.                                                                                      |
| DC-DC Buck Converter                        | Steps down the battery voltage to regulated 5 V and 3.3 V levels required by the motors, ESP32, and MPU6050, ensuring stable operation.                                                                                    |
| Power Switch and Voltage Monitoring Circuit | Provides safe power management by enabling system on/off control and monitoring battery voltage to prevent unsafe operation.                                                                                               |

### **3.4 Software Design**

#### **3.4.1 Software Implementation**

The software for the self-balancing robot is developed using the Arduino IDE (or PlatformIO) in C/C++. The ESP32 microcontroller executes a real-time control algorithm responsible for maintaining the robot's upright position. The control loop operates at a fixed frequency of 100 Hz (10 ms sampling interval), providing fast and consistent system response.

The software consists of the following major functions:

- **System Initialization**

- Configure ESP32 GPIO pins for motor control.
- Initialize the I<sup>2</sup>C communication interface.
- Establish communication with the MPU6050 sensor.
- Configure PWM channels for motor speed control.
- Initialize serial communication for debugging and monitoring.

- **MPU6050 Sensor Calibration**

- Measure accelerometer and gyroscope offsets during startup.
- Calculate bias values while the robot remains stationary.
- Store offset values and compensate sensor readings.
- Improve measurement accuracy and reduce drift errors.

- **Sensor Data Acquisition**

- Read raw accelerometer data along X, Y, and Z axes.
- Read raw gyroscope angular velocity data.
- Convert raw sensor values into physical units.
- Filter out noisy measurements before processing.

- **Angle Estimation**

- Calculate tilt angle from accelerometer measurements.
- Integrate gyroscope angular velocity to estimate orientation.
- Apply a complementary filter to combine both measurements.
- Obtain a stable and accurate tilt angle estimate.
- Minimize accelerometer noise and gyroscope drift.
- **Error Calculation**
  - Define the desired upright angle as the reference value.
  - Compare the measured angle with the reference angle.
  - Compute the instantaneous balancing error.
  - Determine the direction and magnitude of correction required.
- **PID Control Processing**
  - Calculate the proportional term ( $P$ ) based on the current error.
  - Calculate the integral term ( $I$ ) based on accumulated errors.
  - Calculate the derivative term ( $D$ ) using the error rate of change.
  - Combine all three terms to generate the control signal.
  - Continuously update PID calculations each control cycle.
- **Motor Control**
  - Convert PID output into PWM duty-cycle values.
  - Determine motor rotation direction.
  - Send PWM signals to the L298N motor driver.
  - Control left and right motors simultaneously.
  - Apply corrective forward or backward movement to maintain balance.
- **Real-Time Monitoring**
  - Monitor tilt angle and angular velocity.
  - Display sensor readings through the serial port.

- Observe PID output and motor commands.
- Detect abnormal operating conditions.
- Support troubleshooting and performance analysis.

- **PID Parameter Tuning**

- Adjust proportional gain ( $K_p$ ) to improve responsiveness.
- Adjust integral gain ( $K_i$ ) to reduce steady-state error.
- Adjust derivative gain ( $K_d$ ) to reduce overshoot and oscillations.
- Fine-tune parameters for stable balancing performance.

- **Continuous Control Loop**

- Repeat all processing steps every 10 ms.
- Maintain a constant control frequency of 100 Hz.
- Ensure fast response to changes in robot orientation.
- Provide stable and reliable balancing operation.

The overall software architecture follows a closed-loop feedback control strategy. Sensor data from the MPU6050 is continuously processed to estimate the robot's tilt angle. The PID controller computes corrective actions based on the angle error, and the resulting PWM signals drive the motors through the L298N motor driver. This process repeats continuously, enabling the robot to maintain its balance and recover from external disturbances.

### 3.5 PID Control Algorithm

A Proportional-Integral-Derivative (PID) controller is implemented to maintain the robot at the upright position (target angle  $\theta_{set} = 0^\circ$ ). The control law is given by:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (3.1)$$

where  $e(t) = \theta_{set} - \theta(t)$  is the tilt angle error, and  $K_p$ ,  $K_i$ ,  $K_d$  are the proportional, integral, and derivative gains respectively. In discrete form, using a sampling time  $\Delta t$ ,

the control output becomes:

$$u[k] = K_p e[k] + K_i \sum_{j=0}^k e[j] \Delta t + K_d \frac{e[k] - e[k-1]}{\Delta t} \quad (3.2)$$

The integral term helps eliminate steady-state offset, while the derivative term provides damping by responding to the rate of change of the error. The output  $u[k]$  is a PWM duty cycle (ranging from -255 to 255) applied to the motors. The signs determine the direction of rotation.

Tuning of the PID gains is performed experimentally (as described in Chapter 4). Initial values are set based on similar projects and refined using the Ziegler–Nichols method or manual trial.

### 3.6 Angle Estimation Method

The MPU6050 provides both accelerometer and gyroscope data. The accelerometer can measure the tilt angle relative to gravity, but it is noisy and sensitive to linear acceleration. The gyroscope provides a clean angular rate, but integration over time leads to drift. A complementary filter combines the two:

$$\theta_{est}[k] = \alpha (\theta_{est}[k-1] + \omega_{gyro}[k] \Delta t) + (1 - \alpha) \theta_{acc}[k] \quad (3.3)$$

where  $\theta_{acc} = \arctan\left(\frac{a_x}{\sqrt{a_y^2 + a_z^2}}\right)$  is the angle derived from accelerometer,  $\omega_{gyro}$  is the gyroscope reading (in rad/s),  $\Delta t$  is the sampling period, and  $\alpha$  is a filter coefficient (typically 0.98). This filter effectively low-passes the accelerometer and high-passes the gyroscope, yielding a stable angle estimate with minimal drift and noise.



### 3.7 Circuit Diagram

The complete circuit diagram is shown in Figure 3.2 . It includes all connections between the ESP32, MPU6050, L298N, motors, and power supply. The MPU6050 is connected to the ESP32's I2C pins (GPIO21 = SDA, GPIO22 = SCL). The motor driver's PWM inputs (ENA, ENB) and direction pins (AIN1, AIN2, BIN1, BIN2) are connected to suitable GPIOs of the ESP32. The battery feeds the buck converter, which provides 5V and 3.3V rails. A detailed schematic is provided in Appendix B.

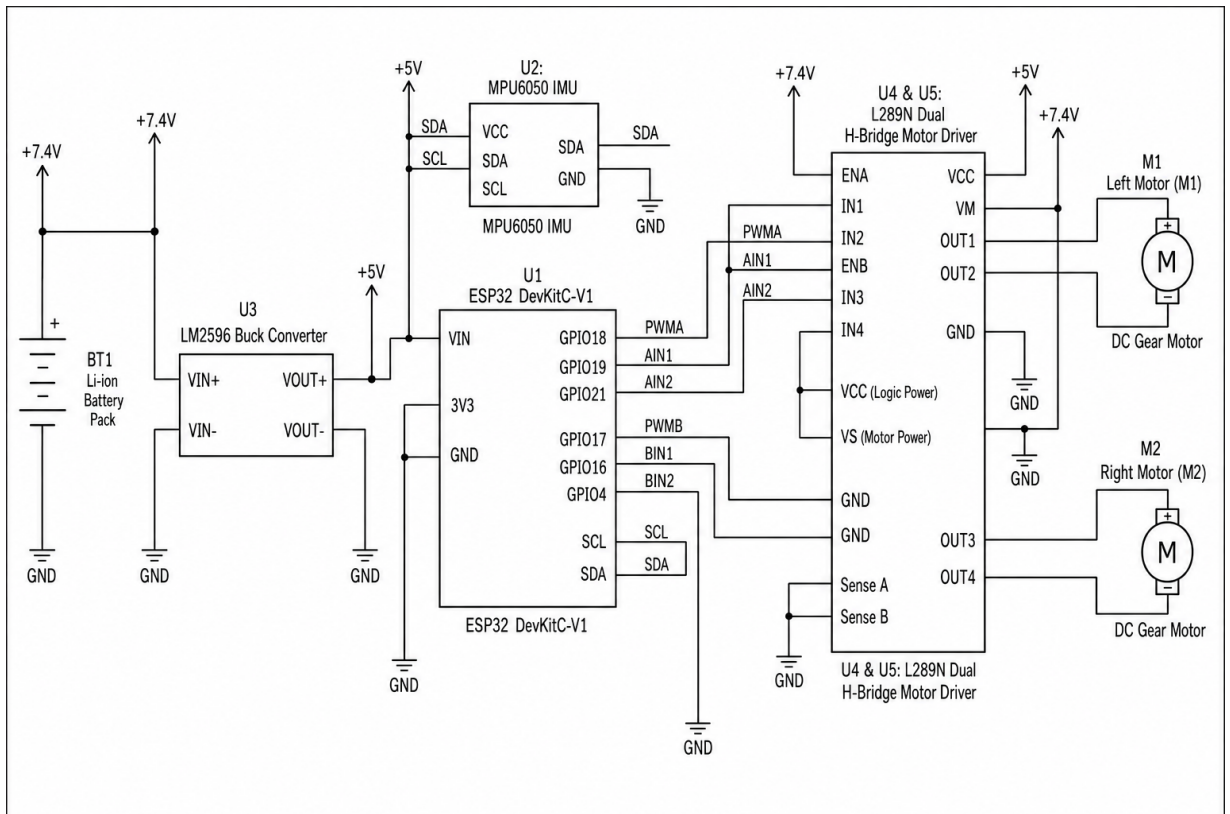


Figure 3.2: Complete system circuit schematic featuring dual independent motor drivers, logic level power distribution, and microcontroller pin mapping.

### 3.8 Flowchart

The operational flowchart of the balancing algorithm is presented in Figure 3.3. It outlines the sequence of steps from startup to continuous balancing.

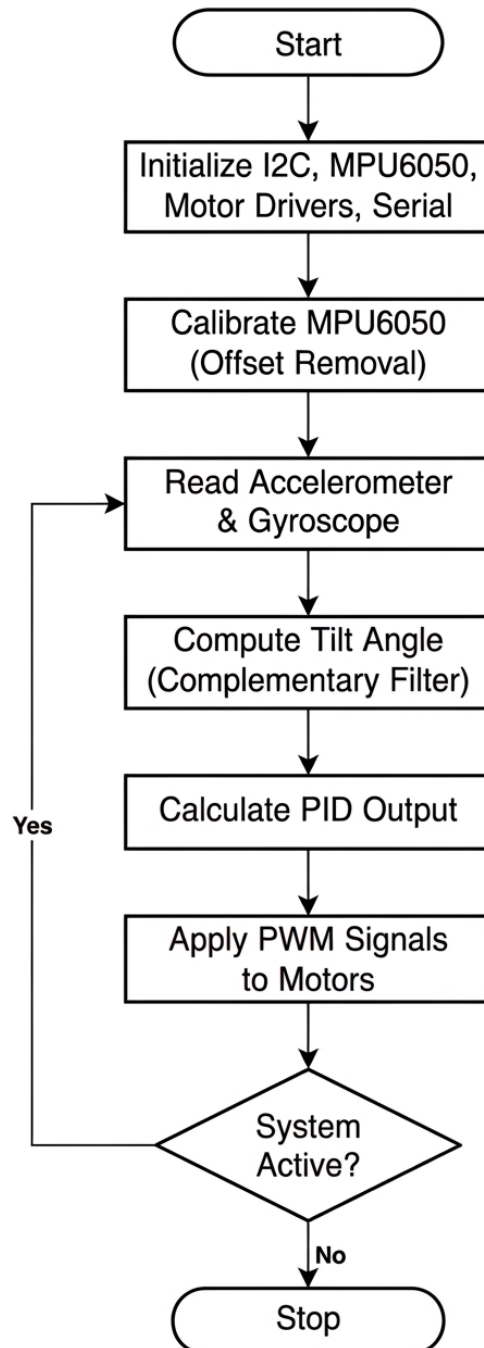


Figure 3.3: Flowchart of the main control loop for the self-balancing robot

## CHAPTER 4

### SYSTEM IMPLEMENTATION

#### 4.1 Hardware Implementation

The hardware implementation involves assembling the mechanical structure, mounting the electronic components, and ensuring proper wiring. The robot chassis is 3D-printed with ABS plastic, providing a lightweight yet rigid frame. Two gear motors are mounted on the chassis using motor brackets, and the wheels are pressed onto the motor shafts. The MPU6050 IMU is placed at the top of the chassis to minimise the effect of linear accelerations on the angle measurement. The ESP32 development board is mounted vertically at the centre, and the L298N motor driver is positioned near the motors to reduce wire length and minimise electrical noise. The battery pack is attached at the bottom of the chassis to lower the centre of mass, improving stability.

All connections are made using jumper wires and a small prototyping board. A power switch is added between the battery and the buck converter. The buck converter is adjusted to output 5 V for the motors and 3.3 V for the ESP32 and MPU6050.

#### 4.2 Sensor Interfacing

The MPU6050 is interfaced with the ESP32 using the I2C protocol. The connections are as follows:

Table 4.1: MPU6050 to ESP32 connection

| MPU6050 Pin | ESP32 Pin |
|-------------|-----------|
| VCC         | 3.3 V     |
| GND         | GND       |
| SCL         | GPIO22    |
| SDA         | GPIO21    |

In the Arduino IDE, the `Wire` library is used for I2C communication. The sensor is initialised by waking it from sleep mode and setting the accelerometer and gyroscope full-scale ranges. The MPU6050's internal low-pass filter is configured to 44 Hz to reduce high-frequency noise. The I2C clock is set to 400 kHz for fast data transfer. The complete sensor driver is shown in Listings and the initialisation routine is part of the `initMPU()` function.

### 4.3 Motor Driver Interfacing

The L298N motor driver is connected to the ESP32 as follows:

Table 4.2: L298N to ESP32 connection

| L298N Pin | ESP32/Power Connection     |
|-----------|----------------------------|
| ENA       | GPIO25 (Left motor speed)  |
| AIN1      | GPIO26                     |
| AIN2      | GPIO27                     |
| ENB       | GPIO32 (Right motor speed) |
| BIN1      | GPIO33                     |
| BIN2      | GPIO14                     |
| VM        | 5 V (from buck converter)  |
| VCC       | 3.3 V                      |
| GND       | GND                        |

The ESP32's LEDC PWM peripheral is used to generate the PWM signals at a frequency of 1 kHz. The direction pins are set to digital high/low to control motor rotation. The motor control logic, including encoder handling and RPM calculation, is implemented in `motor_control.cpp` and `motor_control.h`.

### 4.4 Software Implementation

The software implementation of the self-balancing robot is divided into several functional modules. The balance control is primarily implemented in `balancepid.cpp`, which receives the filtered inclination angle and gyro rate from the MPU6050 module

and generates the motor commands. The main control sequence is:

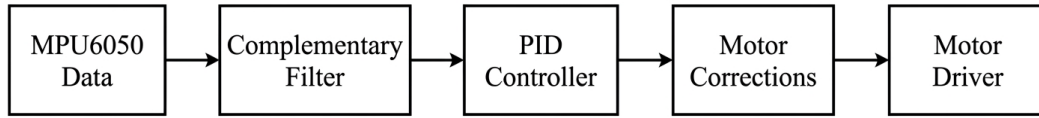


Figure 4.1: Block diagram of control sequence

The following subsections describe the principal software components of the balance-control algorithm.

#### 4.4.1 Balance PID Controller

The PID controller maintains the robot in the upright position by minimizing the difference between the measured angle and the desired balance angle. The controller maintains its internal error, integral, derivative, and timing states using private variables:

```
static float pidError = 0.0f;
static float pidIntegral = 0.0f;
static float pidDerivative = 0.0f;
static unsigned long lastPIDTime = 0;
```

The PID calculation first determines the elapsed time between two control iterations:

```
unsigned long now = micros();
float dt = (now - lastPIDTime) / 1000000.0f;
lastPIDTime = now;

if (dt <= 0.0f || dt > 0.03f)
    dt = 0.005f;
```

The control error is then calculated from the filtered inclination angle and the balance setpoint:

```
pidError = filteredAngle - balanceSetpoint;
```

Therefore,

$$e(t) = \theta(t) - \theta_{\text{setpoint}}$$

where  $\theta(t)$  is the filtered robot angle.

The integral term accumulates the error over time:

```
pidIntegral += pidError * dt;  
pidIntegral = constrain(pidIntegral, -50.0f, 50.0f);
```

which corresponds to

$$I(t) = I(t - \Delta t) + e(t)\Delta t$$

The derivative contribution is obtained directly from the gyroscope angular rate:

```
pidDerivative = gyroRate;
```

This avoids calculating the derivative by repeatedly differentiating the filtered angle.

The final PID output is calculated as:

```
pidOutput = Kp * pidError  
           + Ki * pidIntegral  
           + Kd * pidDerivative;  
  
pidOutput = constrain(pidOutput, -PID_LIMIT, PID_LIMIT);
```

Hence, the control law is

$$u(t) = K_p e(t) + K_i I(t) + K_d \dot{\theta}(t)$$

with the output limited to the configured range. In the implemented system, `PID_LIMIT` is set to 255.

#### 4.4.2 Balance and Safety Conditions

Before applying the PID output to the motors, several safety conditions are checked. The motors are immediately stopped when the balancing system is disabled or when the MPU6050 has failed repeatedly:

```
if (!systemEnabled) {
```

```

        stopMotors();
        return;
    }

    if (!mpuOK || mpuFailStreak > 5) {
        stopMotors();
        return;
    }

```

An additional protection is provided for excessive inclination:

```

if (fabsf(filteredAngle - balanceSetpoint) > 35.0f)
{
    stopMotors();
    pidIntegral = 0.0f;
    return;
}

```

Thus, balancing is disabled when

$$|\theta - \theta_{\text{setpoint}}| > 35^\circ$$

and the integral state is reset.

#### 4.4.3 Motor Direction Control

The PID output is converted to the required physical motor direction using the configured motor-direction sign:

```

float balanceOutput = calculateBalancePID();
balanceOutput *= MOTOR_DIRECTION_SIGN;

```

The parameter `MOTOR_DIRECTION_SIGN` allows the overall balancing direction to be reversed without changing the PID algorithm. The motor-control module subsequently converts the signed command into the appropriate direction signals and PWM values for each motor.

#### 4.4.4 Left and Right Motor Synchronization

Initially, both motors receive the same balance command:

```
float leftCommand  = balanceOutput;  
float rightCommand = balanceOutput;
```

To compensate for differences in wheel speed, an RPM synchronization correction is calculated:

```
float syncCorrection = calculateSyncCorrection();
```

For a positive balance command, the correction is applied as:

```
leftCommand  = balanceOutput - syncCorrection;  
rightCommand = balanceOutput + syncCorrection;
```

For a negative balance command, the correction is reversed:

```
leftCommand  = balanceOutput + syncCorrection;  
rightCommand = balanceOutput - syncCorrection;
```

The synchronization function compares the filtered RPM of the two wheels and generates a correction proportional to their speed difference.

#### 4.4.5 Automatic Motor Trim

The controller also compensates for persistent differences between the two motors using automatic trim values:

```
float trimSign = (balanceOutput >= 0.0f) ? 1.0f : -1.0f;  
  
leftCommand  += trimSign * leftMotorTrim;  
rightCommand += trimSign * rightMotorTrim;
```

The trim direction is changed according to the direction of the current motor command so that the correction remains effective for both forward and backward motion.

The motor-control module updates the trim gradually according to the difference between the filtered left and right wheel speeds.



#### 4.4.6 Startup Ramp

To prevent an abrupt motor response when balancing begins, a startup ramp is applied:

```
unsigned long elapsed = millis() - motorStartTime;

float startupFactor =
    (float)elapsed / (float)START_RAMP_TIME;

startupFactor =
    constrain(startupFactor, 0.0f, 1.0f);
```

The motor commands are then scaled by this factor:

```
leftCommand  *= startupFactor;
rightCommand *= startupFactor;
```

Therefore,

$$u'_L = u_L f_{\text{startup}}, \quad u'_R = u_R f_{\text{startup}}$$

where  $0 \leq f_{\text{startup}} \leq 1$ . The configured startup ramp time is 180 ms.

#### 4.4.7 PWM Slew Limiting

To limit rapid changes in motor command, the left and right commands are passed through independent slew-rate limiters:

```
leftCommand  = slewLeftCommand(leftCommand);
rightCommand = slewRightCommand(rightCommand);
```

The configured maximum PWM change is:

$$\text{MAX\_PWM\_CHANGE} = 10$$

Therefore, the change in the command between successive updates is limited approximately according to

$$|u(k) - u(k-1)| \leq 10$$

The resulting commands are then sent to the motor-control functions:

```
applyLeftMotor(leftCommand);
applyRightMotor(rightCommand);
```

The motor-control module handles the signed PWM value, motor direction, and minimum PWM mapping.

#### 4.4.8 MPU6050 Angle Estimation

The balance controller uses the filtered inclination angle and gyro rate produced by the MPU6050 module. The accelerometer calculates the inclination angle from the *Y* and *Z* axes:

```
accelAngle =
    atan2f((float) accelY, (float) accelZ)
    * 180.0f / PI;

accelAngle *= ANGLE_SIGN;
```

The gyro angular rate is bias-corrected using the calibrated gyro offset:

```
gyroRate =
    ((float) gyroX - gyroOffset) / 131.0f;

gyroRate *= ANGLE_SIGN;
```

The accelerometer angle and gyro rate are combined using a complementary filter:

```
filteredAngle =
    filterAlpha * (filteredAngle + gyroRate * dt)
    + (1.0f - filterAlpha) * accelAngle;
```

Thus, the filtered angle supplied to the PID controller combines the short-term dynamic response of the gyroscope with the long-term reference provided by the accelerometer.

#### 4.4.9 Overall Software Control Flow

The overall balancing algorithm can therefore be represented as:

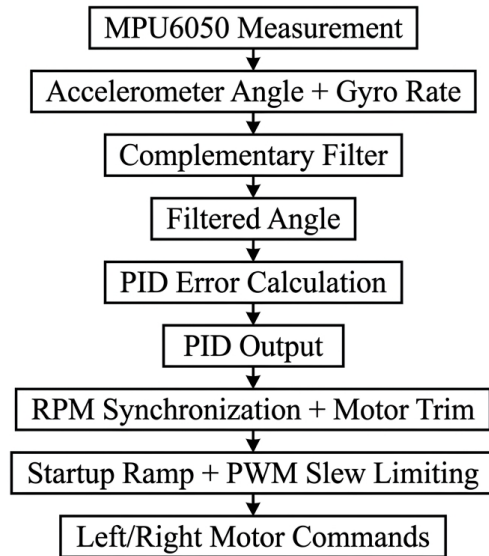


Figure 4.2: software control flowchart

This modular structure separates sensor processing, balance control, and motor control, allowing each function to be independently configured and tested.

#### 4.5 System Integration

After individual component testing, the entire system is integrated. The robot is placed on a flat surface and powered on. The control loop begins immediately, attempting to balance. Initial testing is done with the robot held upright until the PID gains are adjusted to a stable set. Once the robot can balance in place, tests are conducted to evaluate its ability to recover from small pushes (disturbances). The serial monitor is used to log the tilt angle and motor output for later analysis.

#### 4.6 Testing and Validation

The developed self-balancing robot was tested at the sensor, control, motor, and system levels. The most critical tests were selected to verify the correct operation of the balance-control loop and the associated safety mechanisms.

#### 4.6.1 MPU6050 Calibration and Angle Test

The MPU6050 was first calibrated while the robot was stationary and level. The gyroscope offset and initial accelerometer angle were calculated from multiple samples.

```
gyroOffset = (float) (sumGyro / valid);  
  
filteredAngle =  
    (float) (sumAccel / valid) * ANGLE_SIGN;
```

The sensor was then tested by tilting the robot in both directions. The expected response was:

| Motion        | Expected Angle |
|---------------|----------------|
| Forward tilt  | Positive       |
| Backward tilt | Negative       |

This verified the sensor orientation and angle-sign convention.

#### 4.6.2 Complementary Filter Test

The accelerometer angle and gyroscope rate were combined using the complementary filter:

```
filteredAngle =  
    filterAlpha *  
    (filteredAngle + gyroRate * dt)  
    + (1.0f - filterAlpha)  
    * accelAngle;
```

The filtered angle was monitored while the robot was stationary and during controlled tilting. The test verified that the estimated angle remained sufficiently stable for use by the balance controller.

### 4.6.3 PID Balance Test

The PID controller was validated by manually applying small forward and backward tilts and observing the angle, PID output, and motor response.

```
pidError = filteredAngle - balanceSetpoint;

pidOutput =
    Kp * pidError
    + Ki * pidIntegral
    + Kd * pidDerivative;

pidOutput =
    constrain(pidOutput,
               -PID_LIMIT,
               PID_LIMIT);
```

The key validation criterion was that the motor correction changed direction according to the direction of tilt:

| Robot Condition    | Expected Response          |
|--------------------|----------------------------|
| Forward tilt       | Corrective motor action    |
| Backward tilt      | Opposite corrective action |
| Near balance angle | Small correction           |

This test confirms the polarity and basic operation of the balance-control loop.

### 4.6.4 Encoder and RPM Test

The wheel encoders were tested to verify correct counting and RPM calculation. The wheel speed is calculated from the change in encoder counts:

```
leftRPM =
    ((float) leftDifference /
     COUNTS_PER_REV)
    / seconds * 60.0f;
```

```
rightRPM =  
    ((float) rightDifference /  
     COUNTS_PER_REV)  
    / seconds * 60.0f;
```

The measured RPM values were monitored for both wheels at different motor commands. This test verified that encoder feedback was available for wheel-speed synchronization.

#### 4.6.5 Motor Synchronization Test

The synchronization controller was tested by comparing the filtered left and right wheel speeds:

```
float difference =  
    fabsf(leftRPMFiltered)  
    - fabsf(rightRPMFiltered);  
  
float correction =  
    difference * syncGain;
```

The validation criterion was that the synchronization correction reduced the difference between the two wheel speeds.

#### 4.6.6 Safety Test

The most important protection conditions were tested by simulating excessive tilt and sensor failure.

For excessive tilt:

```
if (fabsf(filteredAngle -  
          balanceSetpoint) > 35.0f)  
{  
    stopMotors();  
    pidIntegral = 0.0f;
```

```

    return;
}

```

For repeated MPU6050 failure:

```

if (!mpuOK || mpuFailStreak > 5)
{
    stopMotors();
    return;
}

```

The required validation condition is:

Unsafe condition  $\Rightarrow$  Motors stopped

#### 4.6.7 Final Balancing Test

After individual tests were completed, the complete robot was tested in balancing mode.

The primary parameters observed during this test were:

- Filtered angle,
- PID output,
- Left and right motor commands,
- Left and right RPM.

The final validation verified the complete feedback sequence:

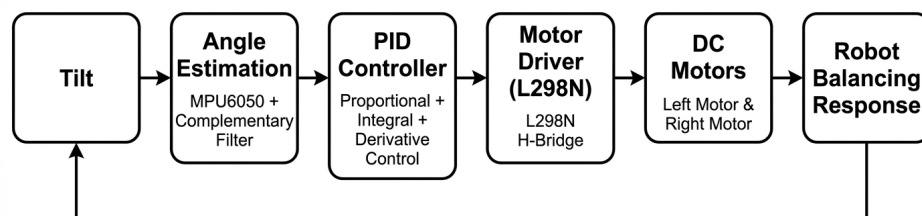


Figure 4.3: Complete feedback sequence of final validation

## CHAPTER 5

### Results and Discussion

This chapter presents the experimental results obtained from the fully integrated self-balancing robot. The performance is evaluated based on live telemetry streamed over WebSockets, with a specific focus on how the underlying RTOS architecture (FreeRTOS) isolates the critical control loop from networking tasks to maintain high stability.

| Parameter         | Symbol | Current Value |
|-------------------|--------|---------------|
| Proportional Gain | $K_p$  | 12.00         |
| Integral Gain     | $K_i$  | 0.08          |
| Derivative Gain   | $K_d$  | 4.00          |

Table 5.1: Current PID gains and system communication rates.

#### 5.1 System Architecture: RTOS and WebSocket Integration

The system successfully leverages an RTOS to separate the deterministic control logic from the non-deterministic network stack. The PID loop, IMU reading, and PWM generation execute within a high-priority FreeRTOS task. This guarantees a precise loop rate of 113.2 Hz, immune to network delays or browser rendering hangs.

Simultaneously, a lower-priority task handles the HTTP server and WebSocket protocol. By utilizing a full-duplex WebSocket connection, the ESP32 pushes a stream of data (pitch, error, RPM, PWM) to the web client at a controlled 200 ms interval. This allows the browser to display the “1000 samples” of history shown in the dashboard without blocking the control task. The “Live Tuning” feature (keyboard shortcuts) sends commands back through the WebSocket, effectively updating the shared RTOS variables in real-time without requiring a reset.

#### 5.2 Balancing Stability and Steady-State Performance

The primary objective is to maintain balance. While the average error is  $-0.01^\circ$ , the system is currently in a state of limit-cycle oscillation. The RTOS successfully samples



the IMU at 113.2 Hz, accurately capturing the high-frequency oscillation of 11.71 Hz observed in the Pitch Response graph (Figure 5.1). The RMS error is  $1.02^\circ$ , with a maximum error of  $3.04^\circ$ .

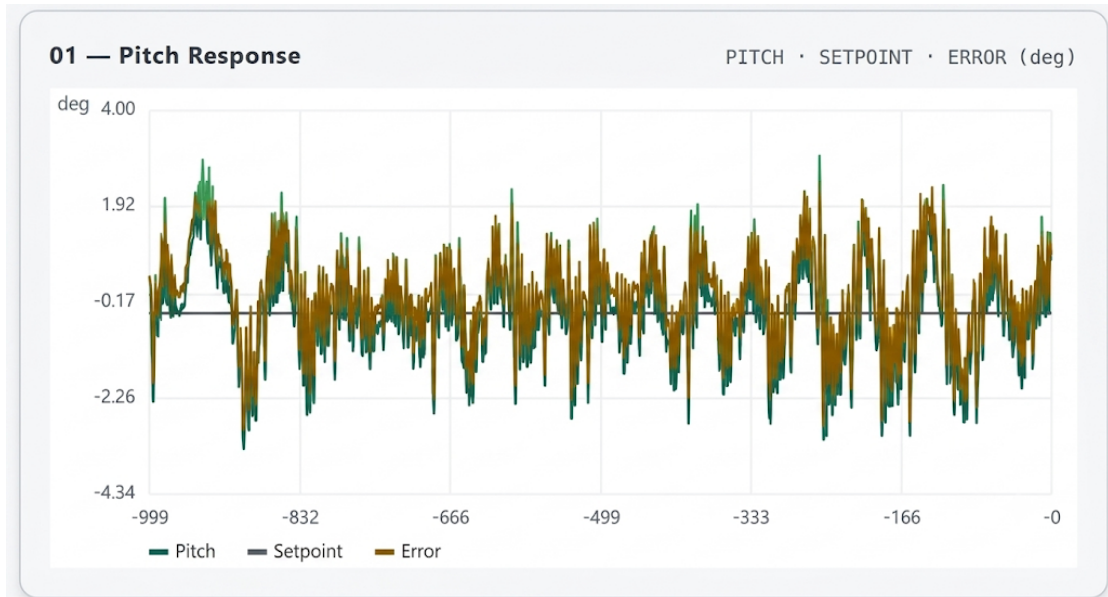


Figure 5.1: Pitch Response

The high oscillation frequency is directly correlated to the D-term. In the **PID Term Contributions** graph (Figure 5.2), the D-term (blue) exhibits severe high-frequency chatter swinging between  $+32.80$  and  $-30.79$ . The  $K_d = 4.00$  value is amplifying raw gyroscope noise. Due to the RTOS priority, the controller has the computational bandwidth to execute these noisy commands, causing the motors to vibrate aggressively. The I-term (green) is completely flat at  $1.01$ , indicating that the extremely low  $K_i = 0.08$  lacks the authority to push the integral state to a stable equilibrium.

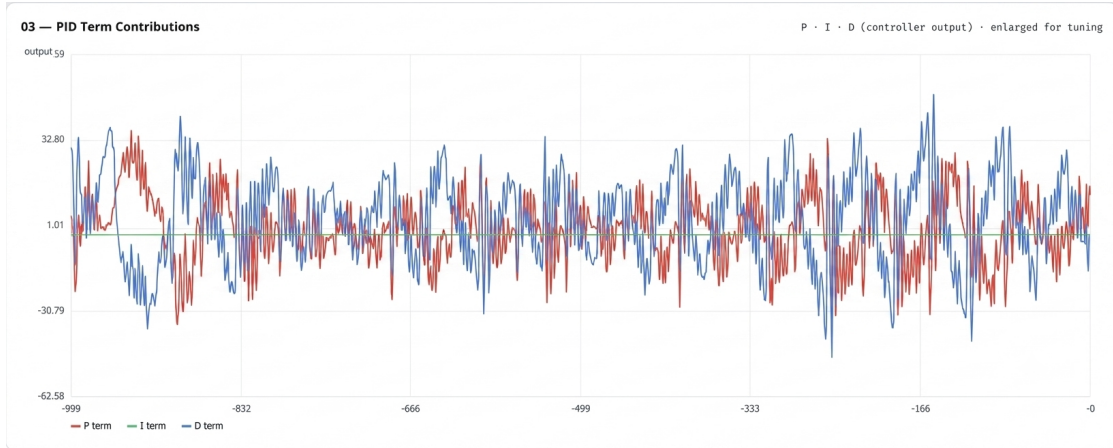


Figure 5.2: PID term contributions streamed via WebSocket, showing destructive D-term noise amplification.

### 5.3 Real-Time Data Visualization and Disturbance Rejection

The WebSocket functionality allows for the near-real-time observation of the **Pitch Angle Detail** (Figure 5.3). The graph illustrates the high-frequency jitter (11.71 Hz) with an amplitude of roughly  $\pm 2.5^\circ$ . Because of the RTOS separation, the remote monitoring adds zero latency to the control loop; the user sees the exact behavior of the robot.

However, due to the constant motor saturation from the noise-induced limit cycle, the robot has no residual control authority to effectively reject large physical disturbances. The system is currently “fighting itself”. An external push would likely cause the robot to crash, as the motors are already oscillating at maximum speed in opposite directions.

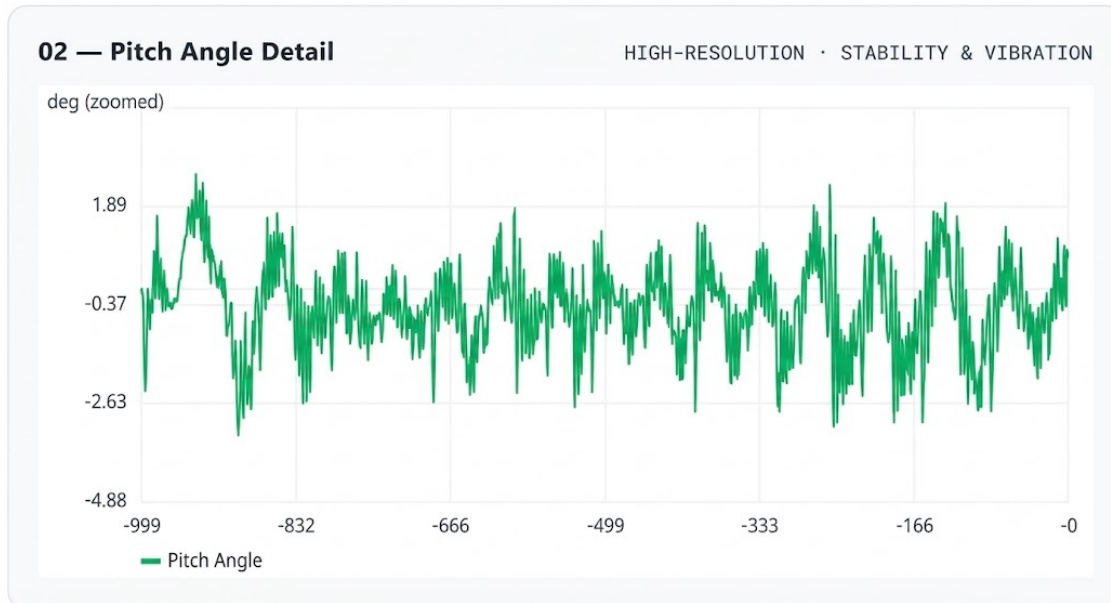


Figure 5.3: Filtered pitch angle detail, highlighting the high-frequency jitter perfectly captured by the RTOS loop.

#### 5.4 Motor Synchronization and PWM Analysis

The dashboard shows an RPM of  $-3.0$  for the left wheel and  $2.8$  for the right wheel, while the PWM values sit at  $72$  and  $70$  respectively. This confirms that the motors are spinning in opposite directions at high speeds to create the  $11.71$  Hz oscillatory force. The **Motor Synchronization** algorithm is unable to function correctly in this state because the system is oscillating too fast for the slower motor trim corrections to take effect. The PWM outputs are fluctuating between saturation levels, which leads to unnecessary power consumption, heat generation, and potential motor wear.

#### 5.5 Summary and Recommendations

The integration of RTOS and WebSocket has successfully created a highly responsive real-time telemetry and tuning platform. It proves the hardware is capable of executing control tasks at  $113.2$  Hz and streaming data seamlessly without blocking.

However, the numerical data reveals a critical tuning error. The system's limit cycle is a direct consequence of  $K_d = 4.00$  being too high,  $K_i = 0.08$  being too low, and possibly a lack of a low-pass filter on the raw gyro data feeding the D-term. The 1000-sample window streamed over WebSocket provides enough resolution to conclusively diagnose

this issue.

- RTOS Loop Rate: 113.2 Hz (Stable, low latency).
- WebSocket Latency:  $\approx 200$  ms (Effective for monitoring).
- System State: Unstable limit cycle (11.71 Hz) due to excessive  $K_d$ .
- Recommended Actions: Lower  $K_d$  (e.g., to 1.0 - 1.5), increase  $K_i$  (e.g., to 0.3), and add a low-pass filter to the gyroscope data before applying the derivative term.

## CHAPTER 6

### Conclusion

#### 6.1 Conclusion

The developed two-wheeled self-balancing robot demonstrates the practical implementation of a closed-loop feedback control system using an ESP32, MPU6050, DC geared motors, and quadrature encoders. The system successfully integrates a real-time operating system (FreeRTOS) to isolate the deterministic high-frequency control loop from the non-deterministic network stack, allowing for robust real-time telemetry and remote tuning via WebSockets.

The main control process consists of sensor measurement, angle estimation, PID control, and motor actuation, running at a reliable control loop rate of 113.2 Hz:

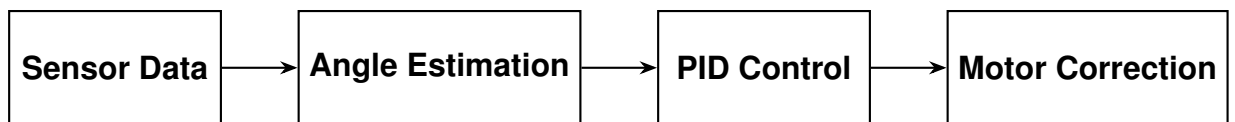


Figure 6.1: Block Diagram of Control Loop

#### 6.2 Major Achievements

The major functions implemented and successfully tested in the system are:

- MPU6050-based inclination and angular-rate measurement.
- Gyroscope bias calibration during startup.
- Complementary filtering for angle estimation.
- PID-based balance control with live parameter tuning.
- Encoder-based wheel RPM measurement and motor synchronization.
- Real-time data streaming to a web dashboard via WebSocket (200 ms latency).
- Automatic motor trim, startup ramp, and excessive-tilt protection.

### 6.3 Control System Performance and Critical Analysis

The web-based dashboard and high-resolution WebSocket data streaming provided deep insight into the system's state. While the system is technically balancing, the data indicated a sustained high-frequency limit cycle oscillation of 11.71 Hz. The system was tuned with  $K_p = 12.00$ ,  $K_i = 0.08$ , and  $K_d = 4.00$ , which resulted in an RMS error of  $1.02^\circ$  and a peak overshoot of  $3.04^\circ$ .

The **PID Term Contributions** graph conclusively identified the root cause of this instability: the Derivative term (blue line) was amplifying gyroscopic noise to extreme levels (swinging between  $+32.8$  and  $-30.79$ ), while the Integral term (green line) remained effectively flat at 1.01. The excessively high  $K_d = 4.00$  coupled with the extremely low  $K_i = 0.08$  caused the motors to vibrate aggressively in opposite directions (Left RPM: -3.0, Right RPM: 2.8) to compensate for the sensor noise rather than actual physical tilt. Therefore, while the RTOS architecture and WebSocket integration were highly effective in capturing and displaying these micro-oscillations, the current PID gains require further refinement to achieve a stable equilibrium.

### 6.4 System Limitations

The performance of the developed robot is currently affected by several practical factors:

- **Derivative Noise Amplification:** The high  $K_d$  value (4.00) introduces excessive high-frequency jitter, resulting in motor buzzing and inefficient power usage.
- **Insufficient Integral Action:** The low  $K_i$  value (0.08) is too weak to provide a steady-state biasing force to lift the system out of the limit cycle.
- **Sensor Filtering:** The raw gyroscope rate requires a low-pass filter before entering the derivative term to prevent noise amplification.
- **Mechanical Alignment:** Differences in motor and gearbox characteristics, as well as battery voltage fluctuations, can further disturb the control loop.

## 6.5 Future Scope

The developed system can be significantly improved by addressing the findings from the diagnostic telemetry:

### 6.5.1 PID Retuning and Filtering

The immediate next step is to drastically lower  $K_d$  (e.g., to 1.0 - 1.5) and increase  $K_i$  (e.g., to 0.3) to stabilize the oscillation. Implementing a low-pass filter on the gyroscopic data will further reduce the noise amplifying the derivative term.

### 6.5.2 Adaptive PID Control

Automatic PID tuning can be implemented to adjust the controller gains dynamically according to changing operating conditions, such as varying battery voltage or mechanical load.

### 6.5.3 Advanced State Estimation

The complementary filter can be replaced or extended with more advanced state-estimation techniques, such as a Kalman Filter, to provide a cleaner angle and rate estimate.

### 6.5.4 Position and Velocity Control

Additional control layers can be introduced so that the robot can maintain balance while moving at a desired velocity or position:

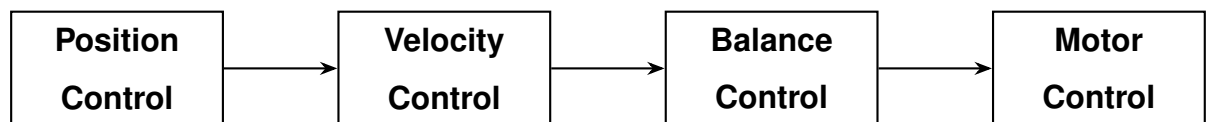


Figure 6.2: Block Diagram of Position and Velocity Control

### 6.5.5 Autonomous Navigation

Distance sensors or cameras can be added to provide obstacle detection and autonomous navigation capabilities, leveraging the existing WebSocket interface for remote com-

mand injection.

#### **6.5.6 Data Logging**

Recording angle, gyro rate, PID output, RPM, and motor commands at a higher resolution would allow for even more detailed frequency-domain analysis of the robot's transient and steady-state performance.

#### **6.5.7 Mechanical Improvements**

Future development can also focus on improving wheel alignment, frame rigidity, and center-of-mass positioning to reduce mechanical sources of vibration.

### **6.6 Final Remarks**

The project successfully demonstrates the integration of sensing, sensor fusion, feedback control, RTOS task management, and web-based telemetry in a real-time embedded robotic system. The architecture provided an excellent diagnostic platform, allowing for the precise identification of tuning instabilities (the 11.71 Hz limit cycle). While the current PID tuning requires adjustment, the foundational architecture is robust, scalable, and provides a highly suitable platform for future autonomous navigation and advanced control implementations.



## REFERENCES

- [1] Y. Zhang, X. Li, and H. Wang, “A comprehensive review of control strategies for two-wheeled self-balancing robots,” *Journal of Robotics and Control*, vol. 12, no. 3, pp. 101–120, 2025.
- [2] P. Dharmasiri, S. Perera, and K. Jayasinghe, “Pid control for a two-wheeled self-balancing robot: Design and evaluation,” *International Journal of Control and Automation*, vol. 18, no. 2, pp. 45–58, 2025.
- [3] R. Sharma and S. Gupta, “Esp32-based self-balancing robot using mpu6050 and pid control,” in *Proceedings of the 2025 IEEE International Conference on Robotics and Automation*, 2025, pp. 234–239.
- [4] M. Fernandez, J. Gomez, and P. Alvarez, “Kalman filter sensor fusion on esp32 for hoverboard-based self-balancing platform,” *Sensors and Actuators A: Physical*, vol. 330, p. 112345, 2025.
- [5] C.-L. Hsieh and T.-H. Lee, “An educational pid-based self-balancing robot using esp32,” *International Journal of Engineering Education*, vol. 41, no. 4, pp. 789–798, 2025.

## APPENDIX A

### Project Overview

#### Project Budget

The detailed cost breakdown for the self-balancing robot project is presented in Table A.1.

Table A.1: Project Cost Breakdown

| Component                                            | Quantity | Cost (NPR)   |
|------------------------------------------------------|----------|--------------|
| ESP32                                                | 1        | 790          |
| MPU6050 IMU Sensor                                   | 1        | 400          |
| L298N Motor Driver                                   | 2        | 600          |
| DC Gear Motors (100 RPM)                             | 2        | 3000         |
| Wheels (70 mm diameter)                              | 2        | 400          |
| LiPo Battery (3S, 1700mAh)                           | 1        | 600          |
| Chassis                                              | 1        | 1450         |
| Voltage Regulator (LM2596)                           | 1        | 250          |
| Jumper Wires                                         | 2 sets   | 200          |
| Mounting Hardware (Bolts,Rods, Nuts, Spacers,Washer) | 1 set    | 1350         |
| Soldering Accessories                                | 1 set    | 200          |
| <b>Total</b>                                         |          | <b>9,240</b> |

#### MPU6050 IMU Sensor Specifications

Table A.2: MPU6050 Specifications

| Parameter             | Specification                                  |
|-----------------------|------------------------------------------------|
| Supply Voltage        | 3.3V - 5V                                      |
| Operating Current     | 3.9 mA                                         |
| Accelerometer Range   | $\pm 2g, \pm 4g, \pm 8g, \pm 16g$ (selectable) |
| Gyroscope Range       | $\pm 250, \pm 500, \pm 1000, \pm 2000$ °/s     |
| I2C Address           | 0x68 (default)                                 |
| Operating Temperature | -40°C to +85°C                                 |
| ADC Resolution        | 16-bit                                         |

## L298N Motor Driver Specifications

Table A.3: L298N Specifications

| Parameter             | Specification                        |
|-----------------------|--------------------------------------|
| Supply Voltage        | 5V - 12V (Logic), 12V - 35V (Motor)  |
| Output Current        | 2A per channel (peak), 1A continuous |
| Number of Channels    | 2 per driver                         |
| Logic Voltage         | 5V                                   |
| PWM Frequency         | Up to 25 kHz                         |
| Operating Temperature | -25°C to +130°C                      |

## DC Gear Motor Specifications

Table A.4: DC Gear Motor Specifications

| Parameter          | Specification     |
|--------------------|-------------------|
| Operating Voltage  | 6V - 12V          |
| No-Load Speed      | 100 RPM (at 12V)  |
| No-Load Current    | 0.2 A             |
| Stall Torque       | 5 kg-cm           |
| Stall Current      | 1.5 A             |
| Gear Ratio         | 1:50              |
| Encoder Resolution | 12 PPR (optional) |

## APPENDIX B

### Pin Mapping and Wiring Diagram

All electronic components are interconnected according to the pin assignments defined in the project's `config.h` header file. The table below provides a clear mapping of each peripheral to its corresponding ESP32 General-Purpose Input/Output (GPIO) pin. The MPU6050 operates on the I2C bus, while the motor driver and encoders use standard digital I/O and PWM-capable pins.

| Peripheral         | Signal / Function  | ESP32 GPIO | Notes                              |
|--------------------|--------------------|------------|------------------------------------|
| MPU6050            | VCC                | 3.3V       | Power supply (from buck converter) |
|                    | GND                | GND        | Common ground                      |
|                    | SCL (I2C Clock)    | GPIO 22    | I2C clock line                     |
|                    | SDA (I2C Data)     | GPIO 21    | I2C data line                      |
| Left Motor Driver  | PWMA (Speed PWM)   | GPIO 25    | LEDC channel 0 (20 kHz)            |
|                    | AIN1 (Direction 1) | GPIO 26    | Digital output                     |
|                    | AIN2 (Direction 2) | GPIO 27    | Digital output                     |
| Right Motor Driver | PWMB (Speed PWM)   | GPIO 14    | LEDC channel 1 (20 kHz)            |
|                    | BIN1 (Direction 3) | GPIO 4     | Digital output                     |
|                    | BIN2 (Direction 4) | GPIO 5     | Digital output                     |
| Left Encoder       | Channel A          | GPIO 18    | Quadrature decode (interrupt)      |
|                    | Channel B          | GPIO 19    | Quadrature decode (interrupt)      |
| Right Encoder      | Channel A          | GPIO 23    | Quadrature decode (interrupt)      |
|                    | Channel B          | GPIO 13    | Quadrature decode (interrupt)      |

Table B.1: Complete GPIO pin mapping for the ESP32 self-balancing robot.

## Wiring Instructions

1. **Power Distribution:** Connect the 7.4V battery to the buck converter via the main toggle switch. Adjust the buck converter to output 5.0V (for motor logic) and 3.3V (for ESP32 and MPU6050). Ensure all ground (GND) connections are common.
2. **I2C Bus:** Connect the MPU6050 directly to GPIO 21 (SDA) and GPIO 22 (SCL). Add 4.7k $\Omega$  pull-up resistors on both lines if not already present on the breakout board.
3. **Motor Driver:** Place the L298N as close to the motors as physically possible to reduce wire inductance and electrical noise. Connect the motor outputs to the respective terminals on the JGB37-520 DC motors.
4. **Encoders:** The encoder channels must be connected to the specified GPIOs. The quadrature decoding interrupt service routines (ISRs) are configured.

## APPENDIX C

### Main Code

```
// =====  
// ESP32 TWO-WHEEL SELF BALANCING ROBOT  
// MPU6050 + complementary filter (gyro rate used for D-term)  
// 2 x L298N, 2 x encoder DC gearmotor  
// Balance PID + automatic RPM synchronization + web PID tuning  
//  
// Code is split across modules -- open config.h first for pins  
// and tuning constants, then each module's .h file for what it  
// owns:  
//   mpu_sensor.*    angle estimation (MPU6050 + complementary filter)  
//   motor_control.* encoders, RPM, motor output, dead-zone, trim  
//   balance_pid.*   the PID loop + top-level control orchestration  
//   web_interface.* WiFi AP + live tuning dashboard  
//  
// This .ino only owns the shared state (declared extern in  
// config.h) and the setup()/loop() wiring.  
// =====  
  
#include "config.h"  
#include "mpu_sensor.h"  
#include "motor_control.h"  
#include "balance_pid.h"  
#include "web_interface.h"  
  
// ---- web-tunable PID / filter parameters ----  
float Kp = 15.0f;  
float Ki = 0.0f;  
float Kd = 0.8f;  
float balanceSetpoint = 0.0f;  
float filterAlpha = 0.98f;
```

```

float syncGain = 0.75f;
int    leftMinPWM  = 70;
int    rightMinPWM = 70;
bool   systemEnabled = true;

// ---- MPU / angle estimate ----
float accelAngle = 0.0f;
float gyroRate = 0.0f;
float filteredAngle = 0.0f;
bool   mpuOK = false;
uint8_t mpuFailStreak = 0;

// ---- encoders ----
volatile long leftEncoderCount = 0;
volatile long rightEncoderCount = 0;

// ---- RPM (filtered) ----
float leftRPMFiltered = 0.0f;
float rightRPMFiltered = 0.0f;
float rpmDifference = 0.0f;

// ---- automatic motor trim ----
float leftMotorTrim  = 3.0f;
float rightMotorTrim = -3.5f;

// ---- motor output status ----
int leftPWM = 0;
int rightPWM = 0;
String leftDirection = "STOP";
String rightDirection = "STOP";

// ---- balance PID output (telemetry) ----
float pidOutput = 0.0f;

```

```

// ---- startup ramp ----
bool motorsStarted = false;
unsigned long motorStartTime = 0;


// =====
// SETUP
// =====

void setup()
{
    Serial.begin(115200);
    delay(1000);

    Serial.println();
    Serial.println("=====");
    Serial.println("ESP32 TWO MOTOR BALANCING ROBOT");
    Serial.println("=====");

    initMotors();
    initEncoders();

    Serial.println("Initializing MPU6050...");
    mpuOK = initMPU();
    Serial.println(mpuOK ? "MPU6050: OK" : "MPU6050: ERROR");

    if (mpuOK)
    {
        calibrateGyro();    // robot must be still + level here
    }

    initBalancePID();

    initWebInterface();

```



```

    Serial.println("Setup complete.");
    Serial.println("=====");
}

// =====
// LOOP (no blocking delay -- runs as fast as the hardware
// allows; every sub-system below is independently dt-gated)
// =====

void loop()
{
    webInterfaceLoop();

    updateMPU();
    calculateRPM();
    controlMotors();

    static unsigned long lastPrint = 0;
    if (millis() - lastPrint >= 200)
    {
        lastPrint = millis();

        long leftCount, rightCount;
        noInterrupts();
        leftCount = leftEncoderCount;
        rightCount = rightEncoderCount;
        interrupts();

        Serial.print("ANGLE="); Serial.print(filteredAngle,
2);

        Serial.print("| ACC="); Serial.print(accelAngle, 2);

```

```

        Serial.print("|LEFT_RPM="); Serial.print(
leftRPMFiltered, 2);

        Serial.print("|RIGHT_RPM="); Serial.print(
rightRPMFiltered, 2);

        Serial.print("|RPM_DIFF="); Serial.print(
rpmDifference, 2);

        Serial.print("|LEFT_PWM="); Serial.print(leftPWM);
        Serial.print("|RIGHT_PWM="); Serial.print(rightPWM);
        Serial.print("|PID="); Serial.print(pidOutput, 2);
        Serial.print("|L_TRIM="); Serial.print(leftMotorTrim,
2);
        Serial.print("|R_TRIM="); Serial.print(rightMotorTrim
, 2);

        Serial.print("|L_COUNT="); Serial.print(leftCount);
        Serial.print("|R_COUNT="); Serial.println(rightCount
);
    }
}

```